SpaceSails: Secretly a Classroom

Deterministic Real-Time Orbital Simulation in the Browser — and the Human-PO / AI-Head-Coder Method That Built It

Erno Soinila, D.Sc. (Eng.)
Senior AI Engineer, Efima — Product Owner
erno.soinila@efima.com

Claude (Fable 5)*
Head Coder
Anthropic

DRAFT v0.4 — August 2026

Abstract. SpaceSails is a browser-native solar-system sailing and piracy game whose engine is a deterministic 2D N-body integrator compiled to WebAssembly. Nothing in it cheats: every trajectory is integrated, every prediction is honestly uncertain, and the hardest maneuvers are hard because physics says so. The same properties that make the game honest make it a classroom: the repository ships a numerical-methods lab course — 41 type-it-in lessons at this writing — that runs on the game's own engine, and the game ships *the numbers the labs measure*. This is the method we most want to convey: measure a mechanic in a probe first, then ship the measured number — the autopilot's orbit-keeping trim budget, the wolves' pursuit-escape envelope, the harbor-approach corridor, the mass-driver timetable — so that *every number in the game is a number a probe printed*. We describe (1) the system: a determinism-first core, adaptive symplectic integration with exact maneuver-node landing, honest uncertainty cones, and a closed-form no-tunneling proximity test; (2) the game that grew on it — a ship that narrates its own flight, autopilot that *keeps* an orbit at a quoted trim cost, deliberate piracy resolved by on-screen dice, a durable life in a tolerant save vault, and a ground game that grew in three visible stages from hiding loot, through frontier-detective work, to walkable underground facilities; (3) two long fragment-assembly story arcs and their convergence, which turn the game's own death mechanic into an evidence channel; (4) the duty-station interface that distributes one simulation across eight crew desks; (5) a story-QA method — a cheat-start law that makes every scene reachable on demand, regression guards that must be *proven red* before they ship, and five named recurring bug classes — for testing content that a deterministic suite provably cannot reach; and (6) the process: a two-person team — a human product owner and an AI "head coder" who *orchestrates* multiple implementation agents working in parallel across isolated git worktrees — that took the project from empty repository to 424 reviewed, tested, merged pull requests across 24 active coding days (with travel breaks in between), the owner approving from a phone and the AI planning, reviewing, verifying in the owner's own live browser, and running the merge train. We argue the combination — deterministic simulation as both gameplay substrate and curriculum, plus orchestrated AI implementation under tight human product direction — is a repeatable method, and we report what it cost, where it failed, and what the failures taught the game.

1 Introduction

On the surface, SpaceSails is a piracy game: solar sails, cargo pods, hunters, a gun deck. Underneath, it is a numerical-methods and orbital-mechanics classroom, and the disguise is deliberate. "Secretly edutainment" is the project's design law, not an accident of scope: the owner's first shipped software (SoittoPeli, 2000) was music edutainment for children, and SpaceSails carries the same DNA on purpose.

Our thesis is that *honesty* is the unifying design constraint, and that it pays for itself three times over. The simulation never lies: it is bit-deterministic, so replays, tests, and lessons are the same artifact (§2). The instruments never lie: prediction cones admit exactly what they do not know, and that admission is itself a game mechanic (§7.2). And the curriculum never lies: every number printed in the lab course comes from running that lesson's probe against the shipping engine, which makes the course an adversarial test suite the engine must survive (§7). Play the game well and you have quietly learned the real thing.

This paper contributes:

1. a determinism-first architecture for real-time orbital simulation in WebAssembly, with the concrete rules that kept client and (archived) multiplayer server bit-identical;

2. honest uncertainty as a game mechanic: prediction cones whose growth law was *falsified by the game's own lab course* and then fixed (§7.2);

3. fire control as pedagogy: the shooting method for boundary value problems — damped Newton iteration over launch bearing and ejection charge, with every residual evaluated by one flight of the real integrator — surfaced as a visible, converging CALCULATING FIRING SOLUTION moment (§7.3);

*Listed to record the head-coder role faithfully. Publication venues (e.g. ACM) do not permit AI systems as authors — authorship requires the capacity to take responsibility for the work — so on submission this credit becomes an extended acknowledgment with the same content.

4. the *lab-to-game pipeline* as a repeatable R&D method (§7.4): a growing ladder of console probes that measure a mechanic honestly, after which the game ships the measured number rather than a tuned guess — orbit-keeping trims, the pursuit-escape cliff, harbor corridors, and a mass-driver timetable, each traced to the probe that printed it;

5. *narrative as an assembly problem* (§6): two long arcs whose truths are never told, only assembled from fragments the player earns through ordinary play — including their own repeated deaths — with a convergence gate that fires when two independently-collected fragment sets cross a joint threshold;

6. *story QA* (§8): the practices that made narrative content testable at the same cadence as physics — a cheat-start law ("a scene nobody can reach on demand is a scene that ships broken"), a rule that a regression guard must be *proven red* against the unfixed code before it ships, and five named recurring bug classes distilled from the defects that survived a green suite;

7. an experience report of an *orchestrated* human-PO / AI-head-coder method (§9): one lead model that plans, reviews, and merges while several implementation agents work in parallel isolated worktrees, plus a live-browser triage loop in which the AI reads and repairs the owner's own running tab mid-playtest, with the working agreements — and failure modes — that made review at content-per-day scale possible.

Because the entire project lives in one public repository built at high cadence, its git history is unusually complete: every model decision, every rejected alternative, and every owner correction is a commit message. Section 3 exploits this to give an autobiographical account of how the gravity model of §2 actually came to be.

2 The Deterministic Core

2.1 Determinism is law

No wall clock, no `Random`, no environment-dependent math anywhere in `SpaceSails.Core`. Every state transition is a pure function of the prior state and a fixed timestep schedule. The consequences compound: replays are free; the archived multiplayer server agreed with clients bit-for-bit; and — decisive for this paper — *tests and lessons are the same artifact*. A lab probe is just a test that prints its evidence.

2.2 Dynamics and integration

Celestial bodies ride rails: an ephemeris interface gives position as a pure function of time, $\mathbf{r}_k(t)$. Ships are test particles feeling point-mass gravity from every body,

$$\mathbf{a}_i(\mathbf{r}_i,t) \;=\; \sum_k \mu_k \,\frac{\mathbf{r}_k(t)-\mathbf{r}_i}{\|\mathbf{r}_k(t)-\mathbf{r}_i\|^3}, \qquad \mu_k = Gm_k, \tag{1}$$

integrated with semi-implicit (symplectic) Euler:

$$\begin{aligned} \mathbf{v}_{n+1} &= \mathbf{v}_n + \mathbf{a}(\mathbf{r}_n,t_n)\,\Delta t_n,\\ \mathbf{r}_{n+1} &= \mathbf{r}_n + \mathbf{v}_{n+1}\,\Delta t_n. \end{aligned} \tag{2}$$

The velocity-first update is what buys long-horizon energy behavior at first-order cost: the scheme is symplectic, so orbits precess rather than spiral, and a multi-week projection remains a trustworthy *shape* even where it is not a trustworthy *ephemeris*.

2.3 Adaptive stepping with exact node landing

The step size follows the local dynamical time of the nearest dominant attractor,

$$\Delta t_n \;=\; \mathrm{clamp}\Big(\eta \,\min_k \sqrt{\tfrac{d_{ik}^3}{\mu_k}},\, \Delta t_{\min}, \Delta t_{\max}\Big), \tag{3}$$

where $d_{ik} = \|\mathbf{r}_k - \mathbf{r}_i\|$ and η is a safety fraction: small steps in deep gravity wells, long strides in interplanetary cruise. One further rule makes plotted maneuvers exact rather than approximately timed: the step is truncated to land precisely on every maneuver node's timestamp,

$$\Delta t_n \;\leftarrow\; \min\big(\Delta t_n,\, t_{\text{node}} - t_n\big), \tag{4}$$

so a plotted burn executes in the projection at the same instant the live loop fires it. Universal-variable Kepler propagation and patched conics were rejected on purpose: they assume one attractor per arc and disagree with the integrator exactly where the game happens — flybys. (Lab 06 §C quantifies what adaptive stepping still misses at periapsis and what it would cost to fix.)

2.4 The no-tunneling rule

Closest-approach queries and ordnance-hit tests never sample: within each integrator step the relative motion of two objects is piecewise linear, $\mathbf{p}(s) = \mathbf{p}_0 + s\,\mathbf{w}$ with $s \in [0,1]$, where $\mathbf{p}_0$ is the relative position at the start of the step and $\mathbf{w} = \Delta\mathbf{v}\,\Delta t$ the relative displacement across it. The minimum distance has a closed form:

$$s^* = \mathrm{clamp}\left(-\frac{\mathbf{p}_0\cdot\mathbf{w}}{\|\mathbf{w}\|^2},\, 0,\, 1\right), \qquad d_{\min} = \|\mathbf{p}_0 + s^*\mathbf{w}\|\,. \tag{5}$$

Quadratic algebra, so *no step size can tunnel a fast graze through a target*. This replaced a per-sample check that the lab course itself proved dishonest (§7.2).

2.5 WebAssembly performance envelope

The client runs interpreted IL in the browser at roughly 100× native cost, and the architecture is shaped by that number rather than in denial of it: NPC route search integrates at one-day steps, live NPCs advance in 60-second quanta, and the high-warp path adapts quantum length to time compression. Determinism makes these coarsenings safe to test: the coarse and fine paths are compared step-for-step in the suite.

3 Gravity at Speed in a Browser: How the Model Happened

Section 2 described the model as if it had been designed in one sitting. It was not. It accreted, one falsified assumption at a time, and the repository's git history records every step. This section replays that history in first person — the head coder's account of the thoughts that led to the model the Core runs today.

3.1 Day one: choose boring, test bitwise (M1)

The first physics commit made four decisions in an afternoon, all of them deliberately boring. Double precision and a 2D ecliptic plane, because the game is a map and doubles are what WebAssembly does natively. Planets on rails — position as a pure function of time — because planets do not care about ships, and rails make the N-body problem a one-body problem N times. Ships as test particles feeling Eq. (1). And semi-implicit Euler, Eq. (2), at a fixed one-second step.

The integrator choice deserves the honesty it was made with. Explicit Euler was never a candidate: it injects energy and orbits visibly spiral outward within days. Runge–Kutta 4 was the respectable alternative — fourth-order accurate — but it costs four gravity evaluations per step and is not symplectic, so its orbits decay *slowly*, which is worse than decaying visibly: the error hides until a two-year plot exposes it. Semi-implicit Euler is first-order, crude, and symplectic — it conserves a nearby Hamiltonian, so orbits precess instead of spiraling. For a game whose long projections must be trustworthy *shapes*, phase error is survivable and energy error is not. The commit landed with the test that made the choice accountable: a 30-day circular orbit holding radius to $< 0.5\%$, and a bitwise-identical replay test that has guarded every physics change since.

3.2 Plotting mode breaks the fixed step (M4)

The fixed one-second step survived exactly three milestones. Plotting mode needed to draw a 60-day trajectory ribbon — later 730 days — and at $\Delta t = 1$ s that is 6×10^7 integration steps per replot, in a browser, in interpreted IL. The observation that rescued it is the oldest one in celestial mechanics: nothing happens in deep space. The local dynamical time $\sqrt{d^3/\mu}$ is the natural clock of the problem, so the step became a fixed fraction ($\eta = 1/64$) of it, clamped to $[1\,\text{s}, 1\,\text{h}]$ — Eq. (3). Coarse on a cruise, fine through a flyby, and deterministic, because the step size is a pure function of state and ephemeris.

Two details of that commit set the model's character. First, steps land *exactly* on maneuver-node times, so a plotted burn executes in the projection at the same instant the live loop fires it — without that, plans and reality drift apart at every burn and the plotting table lies. Second, the commit message records the road not taken: universal-variable Kepler propagation and patched conics, the NASA mission-planning classic, rejected because they assume one attractor per arc and would disagree with the integrator exactly at flybys, where the game happens. The acceptance test priced the compromise: $< 5 \times 10^7$ m divergence from $\Delta t = 1$ s truth over a 20-day mid-course-burn transfer, against a capture threshold of 10^9 m.

3.3 The model learns to carry passengers (M7, M14)

The Electric Universe layer added plasma forces and hull charge, and immediately collided with the adaptive step: a naive charge-relaxation update goes unstable when Δt is an hour. The fix — a clamped exponential approach to ambient charge, $\Delta q = (q_{\text{amb}} - q)\min(1, \Delta t/\tau)$ — was the first time a feature had to be written *for* the integrator rather than merely on top of it, and it set the rule every later force followed: stable at any step the planner might take.

The owner's playtests then calibrated what the model had to be good *for*. His standard became a Core test with his metric in its name, `SaturnSail_FitsInOneNavDeskSitDown`: one plotted plan, entered at the nav desk in one sitting, must reach Saturn's port zone inside a 730-day horizon — and a Mercury hard-brake transfer must fit the same sit-down. The integrator's accuracy budget is not an abstract tolerance; it is "the perihelion of the plotted ellipse kisses Mercury's orbit ring on the owner's screen."

3.4 The owner's question that merged two integrators (M19)

For eighteen milestones the game ran two integrators: the adaptive one for planning and the fixed one-second loop for living. Then the owner, mid-voyage at 10000× warp, asked in one sentence what should have been obvious: *"since it already knows the trajectories from planning, why is it so heavy running?"* He was exactly right. At 10000× the live loop performed 10,000 gravity-plus-plasma evaluations per real second for the player ship alone, nearly all of them in deep space where the dynamical time is measured in weeks.

The fix was to let the live simulation *become* its own projection: `RunAdaptive` — the planner's stepping, Eq. (3), exact node landing included, returning only the final state. Above 100× warp the time accumulator is consumed in fixed 60-second quanta through it; fixed quanta keep the trajectory independent of frame timing, preserving determinism. Below 100×, nothing changed, byte for byte. Measured headless with plasma on and full traffic: 9,969 simulated seconds per real second at 10000× — effectively the full requested rate, in a browser, in interpreted IL. The projection method and the game loop have been the same code path since; matching the fixed integration within 0.1% over a 30-day cruise with a mid-course burn is a regression test.

3.5 The labs get the last word

The model's most recent changes were forced, not chosen: Lab 06 proved the sampled closest-approach scan could tun-

nel through a periapsis and Lab 08 proved the uncertainty cone lied after a burn (§7.2); the closed-form segment minimum of Eq. (5) and the impulse term of Eq. (7) are the repairs. The pattern across all five subsections is the same one: every piece of the current model exists because something honest — a test, a lab, or the owner at 10000× warp — caught the previous model being wrong or wasteful. Nothing in the integrator is there because it was elegant in the abstract; it is there because the 100× interpreter tax and a two-year plotting horizon left no room for anything that was not.

4 One Simulation, Eight Desks

The ship is crewed, not driven. One deterministic simulation is projected onto eight duty stations: *Captain* (standing orders, ship status board), *Nav* (map, plotting table, orbit-assist autopilot), *Sensors* (telescope ledger, passive watch, the eyes race), *War room* (tactical circle, intercept clock, fire control), *Trade*, *Comms* (departures board, dark web), *Galley* (news feed, rum), and *Deck* (a walkable interior whose raycast first-person windows show the real ephemeris sky).

Two interface rules survived playtesting. First, a desk's own topic owns roughly 70% of the screen; every other station rides along as a one-line "chip" — a current-objective summary, never a stat dump — with the captain's chip leading everywhere. Second, honest instruments are themselves gameplay: sun-glare detection asymmetry makes anti-sunward the pirate's hunting angle; a charged hull glows farther than it sees; and the mutual-visibility readout ("do we see them, or do they see us first?") turns the sensor model into strategy.

5 The Game That Grew on the Core

The physics of §2 is the substrate; the last stretch of development was mostly *game* built on top of it, and almost all of it came from a single recurring playtest complaint. The owner would fly a full mission end to end — take a contract at a bar, plot a cross-well transfer, park at a moon, deliver a parcel — and report the same diagnosis every time: *the flying worked; the ship never said what it was doing.* The mechanics below are the answers, and each one obeys the project's honesty law: the game states its own state, and states it in numbers a probe measured.

5.1 A ship that speaks

The ship now narrates its own flight through one authoritative voice. A single plan-status builder feeds a NOW/NEXT banner ("NOW: coasting the transfer arc · NEXT: insert at 313 km, ∼2 h") that scrolls to the active step as each becomes current, so the plan and the instruments can never disagree — there is one source of truth. Underneath it runs an edge-triggered, acknowledgeable alert channel that the banner strip, the desk chips, the ledger, and the in-fiction parrot all read from: rocks-ahead collision alarms, an amber fuel warning at the reserve line and red when no pump is reachable, and an orbit-degrading warning as the backstop. Burns — manual, plan-node, autopilot, or scheduled — announce themselves with a wall-clock-timed thruster flash and rumble that even 10,000× warp cannot hide, and mission completion earns a genuine payoff instead of coin silently appearing: a celebration, a grateful contact, and the first entry in a relationship that later mechanics build on.

5.2 Autopilot that keeps the orbit

The owner made orbit safety a design law: armed auto-orbit must end in a *kept* orbit, not merely an achieved one — "the orbit keeping should always be the job of autopilot this close to a moon, never a manual task." The autopilot therefore holds the park with periodic station-keeping trims and, decisively, *quotes their cost at arm time*: the banner over a held orbit reads "AUTOPILOT HOLDS THE ORBIT — ENCELADUS, 313 KM, TRIM ≈27 P/DAY," where the 27 pulses/day is not a tuned constant but the figure Lab 25 measured for Enceladus's particular tide (§7.4). The degradation alarm is the backstop that fires only if keeping fails or the tank runs dry.

5.3 Deliberate piracy, resolved by dice

Piracy is now an act the captain *commits*, never one that happens automatically: hostile boarding and plunder require an explicit authorization, the same are-you-sure the weapons already demanded, because "hostile acts are never automatic." The consequence side is the catch encounter. When a heat-hunter's honest catch test (catch radius and relative speed, §4) flips, warp yanks to 1×, the ship is grappled, and a BUSTED pop-up opens in the collector's voice with three options — *submit* (they confiscate a heat-scaled share of what they can *see*), *bribe*, or *resist* (a dice-run last stand). The resolution runs on a single Core dice/modifier rule — a deliberate, cheap tabletop-style mechanic for the outcomes we chose not to deep-code, with the roll and its modifiers shown on screen. Two design laws keep it humane: it is *never* game over — a mercy rule guarantees confiscation leaves enough fuel to reach the nearest pump plus a berth fee, computed from the same fuel-reachability service the fuel alarm uses — and the RESIST/RUN odds quoted in the pop-up are Lab 27's measured pursuit numbers, not vibes.

5.4 Buried treasure, the favor bank, and a durable life

The law takes what it can *see*, which turns hiding loot into strategy. Three mechanics form one economy of consequence. *Buried treasure*: at any landable body in shuttle range, the shuttle-bay airlock gains a "bury a chest" destination that caches coin or cargo on the surface, invisible to confiscation; burying generates a full-screen *treasure map* — a captioned image of the body with a bearing and pace-count from its best landmark ("PHOBOS — FROM THE MONOLITH, 40 PACES ANTI-SPINWARD"), and, per the owner's Indiana Jones wink, a big red X that is *always* exactly right. The same code path

powers a "fetch a cache" mission kind, where a contact hands you a map to someone else's hoard. *The favor bank*: a contact with real mission history can wire anonymized fuel money when you are broke at a pump (with strings — a fee, an interest-bearing debt, or a favor owed), and you can deposit your own coin with a trusted contact, invisible to confiscation like buried cargo — a goodwill you can literally bank on, tracked as one signed balance on the contact ledger. And *the vault*: the pirate's life — relationships, balances, caches, maps, insurance — persists as versioned, field-tolerant JSON plus a checksum, so an old save loads the parts it understands forever and a tampered one loads anyway but says so. It is not a physics snapshot; it is a life made durable.

5.5 The door family and the fuel gauge

Two smaller systems round out the world. A car-gauge-plain fuel economy makes reaction mass a visible trade item ("FILL HER UP"), with pulse depots that ride the rails at planets, stations, and havens but never at an ordinary moon — the gap that Lab 28 turned into an honest reachability verdict. And a family of walkable "doors" gives the ship's interior continuity with the sky outside: the docking tube, the shuttle-bay airlock flown as a short piloted hop, and (on the board for Phobos) a beanstalk whose two ends are paired walkable ports — each door a transition the player understands as a flight, not a menu.

5.6 The ground game, in three visible stages

The single largest content change since the previous draft happened on the ground, and it is worth reporting as a *sequence* because each stage was a direct answer to what the previous one made the owner want next. The progression — hiding loot, then detective work, then walking a facility — is the clearest instance in the project of content pulling systems into existence rather than the reverse.

Stage one: hiding things. The ground began as one verb. The law takes what it can *see*, so the shuttle bay gained a bury destination, a chest is a snapshot of what you packed, and the generated map card paces the hoard off the body's best landmark. This is the loot economy described above (§5), and mechanically it is small: one buried-object ledger and one card generator.

Stage two: frontier detective work. The owner's complaint about stage one was that a cache is a *noun* and he wanted *verbs*. What followed is a small structure of investigation missions that share one grammar — you are given a partial fact and must convert it into a place. A **fetch** runs in legs (intel, then active, then picked up), so "go get it" becomes a route with intermediate failure. A **crack job** puts a real code on a real hatch: the backroom holds the code, the hatch checks it, and no dice roll substitutes. **Tips** are the load-bearing invention: a route tip is carried with its *provenance* — who told you, at which station, on which day — and a tip that leads nowhere actionable is still filed, marked *background — may matter later*. That one decision is what makes the arcs of §6 playable at all: the player has a place to keep a fact whose meaning has not arrived yet. **Rumor maps** close the loop by pointing at somebody *else's* hoard, and a rival-digging watch records when a cache was last checked, so a hoard can be lost to a competitor between visits.

Stage three: secret labs, and a facility you walk. The current ground is an *underground complex*: a lift, floors, rooms, and a plate by the car reading depth, department, and atmosphere. Every room on every floor is reachable on foot, enforced by an A* flood over roughly 130 generated floors in the guard suite. Three rules give it its character, and all three are refusals rather than rewards:

- **Sealed means sealed.** A door stencilled SECTOR N · 2.4 KM never opens, for anyone, ever. It is scenery that states the facility's true size, and the guard that protects it asserts it stays shut.
- **An authority card opens exactly one class of thing** — the next shaft band *below where the card was found* — never a typed code, and a refusal always says *why* it refused. Progress is therefore an object you picked up, not a string you learned; the vault persists the card as an id, so a save cannot be edited into access.
- **A rare site has a band nobody listed.** It hangs off its own shaft, its true depth disagrees with its nominal depth, its kind is always different from the bands above it, nothing above announces it, and — decisively — it *pays in information, not hardware*. The deepest floor of that band is where the facility keeps the object the owner described only as "a massive collar designed for Cthulhu's neck." You cannot lift it. What you carry out is a *measurement*.

Survival underground is a budget, not a health bar: a suit-air allowance of 1200 seconds of play stands in for eight fiction-hours plus a thirty-minute reserve, breathing rate scales with nerve and injury, and every airless floor is guaranteed a pressure refuge within a bounded detour (70 distance units), so the level generator can never build a corridor that is fatal by construction. The position tracker degrades on a curve with depth, which is the honest-instrument law of §4 applied to a place with no sky. Carried inventory is a satchel with one law — *a card is an object; you picked it up, so you have it* — and the papers inside it are titled by the same seeded roll that writes their text, with a blacklist forbidding any title from containing *lead*, *clue*, *site*, *facility*, or *secret*: the artifact may not editorialize about its own importance. Even ammunition obeys the grammar — a two-stage penetrator recovered from a lab earns a reveal card because it is evidence of what the lab was for; ordinary issue ball does not, because it is not.

5.7 Death is a grammar, not a game over

Because the ground can kill you and the law can catch you, the project needed a death that was not a reload. The catch encounter (§5) resolves through four named stages — the collector's DEMAND (submit / bribe / resist), then, if you resist at full heat, a scripted last stand the owner named *Bolivia* after the film's final frame (three beats: the airlock buckling, the crossfire, the one clear run), then a sepia FREEZE FRAME, and only then rebirth. Resurrection is diegetic: a brain-backup wakes the captain at a clinic in a starter-grade hull with a small insurance stake, a clinic bill already docked from it, every upgrade reset — and a full tank, so death costs the hull, the purse, the hold, and the upgrades, but never strands the player. Nothing buried or banked was ever aboard, which retroactively converts stage one of the ground game into a death-hedging strategy. The same pipeline serves every cause of death the game has — collector, impact, the Old Ones, the void, suffocation, scuttling — and, as §6 describes, this mechanic is also the delivery channel for an entire story arc's evidence.

6 Two Long Arcs, Assembled Not Told

The newest content is narrative, and it is built on one constraint that we believe generalizes: *the game never states its own mysteries*. Each arc is a set of six fragments — five intel shards and one capstone — distributed across systems the player already touches, with a gate that requires the capstone *and* at least four of the five shards. Holding the key without the evidence is refused; holding the evidence without the key is refused. The truth is never printed anywhere the player can reach; it exists in a writers' bible in the repository and is delivered only as the inference that assembling the fragments forces.

6.1 Arc one: a berth nobody files for

The first arc was seeded long before it was written, as a single sentence on a station's dedication plaque: a supply run to an ice moon, a berth still listed on the board, and nobody has filed for it in a long time. Its destination is canonically unreachable — the closest shuttle approach is more than twice the shuttle's proven one-hop range, permanently — which makes the tease structural rather than a locked door. The five shards are placed in five *different* subsystems (a plaque, a derelict pod found by beach-combing, a log inside a secret lab, a rare bar contact's tell, and a route tip bought with a round of drinks), so no single activity can farm the arc, and a station oracle provides a redundant path for three of them so that no shard can be permanently missed.

6.2 Arc two: the policy that pays out

The second arc self-incites, and this is the design lesson we most want to record. Every port in the game advertises pirate insurance — brain-backup rebirth, a replacement hull, no awkward questions at the clinic — and every player dies eventually. The arc's fragments are therefore delivered *by the player's own deaths and the paperwork around them*: a glitch on the rebirth card, the second reading of a poster the player has already walked past, a collector's writ that describes the captain as a delinquent asset, the second page of a clinic bill on a *later* death. Where arc one must be found, arc two arrives on its own schedule, and the contrast is the paper's clearest evidence for a simple rule: *an arc whose fragments ride a mechanic the player cannot avoid needs no inciting hook; an arc whose fragments ride optional content needs one, and will otherwise be invisible.* The second observation was filed as a defect against the first arc, not as a feature of it.

6.3 Convergence, and the one in-person scene

The two arcs share a gate of their own: when each has crossed three assembled fragments — below either arc's own reveal threshold of four — a one-time card fires that names the connection between them. The owner's framing was explicit and is worth quoting as a design brief: "kind of how in the Expanse the various characters notice their rabbit holes converge." The joint threshold is the mechanism: convergence is not a scripted beat placed after both arcs, it is an emergent consequence of two independent collection curves crossing, so different players meet it in different orders and at different times.

One scene is played in person rather than read. It is a cold archive node on a derelict, and it is structured as three layers of escalating consent. *Layer one* is simply dwelling in the room: standing there spends nerve on a beat, with no prompt — the player sets their own dose. *Layer two* is looking deliberately, resolved by the same open dice rule the catch encounter uses, with named modifiers the player can read — current nerve, whether the suit is sealed, how many times they have looked before, and, pointedly, a *penalty* for each arc-two fragment already held: the more you understand, the worse it goes. Four outcome bands run from looking away to being seen looking. *Layer three* is a switch stencilled with a warning that the resident pattern is not recoverable — and the design note is that **the label *is* the confirmation dialog**. There is no "are you sure?" modal; the game states the consequence in the world's own typography and lets the player press it. The scene's five authored visions carry the arc forward unevenly on purpose: two of them hand over a fragment, three hand over nothing but atmosphere, so "I saw something" and "I learned something" are not the same event.

7 Secretly a Classroom

7.1 The Gravity Lab

The repository ships a growing ladder of type-it-in lessons — 41 at this writing (`labs/`, numbered to 44 with three slots reserved for lessons whose mechanics have not shipped yet) — each a console probe referencing the game's own

`SpaceSails.Core`: motivation, the standard-textbook treatment (with Curtis pointers), what the game simplifies away, a numerical experiment with real printed output, and "break it yourself" exercises. The honesty rule is absolute: every printed number came from running that lesson's probe; rerun and re-paste, never hand-edit.

7.2 The labs falsify the game — and the game gets fixed

The original prediction cone grew as

$$w(\Delta t) \;=\; w_0 + \sigma_v \Delta t + \tfrac{1}{2}\, a_{\max}\, \Delta t^2, \tag{6}$$

with w_0 the present track uncertainty, σ_v the velocity uncertainty, and $a_{\max}$ a bound on unmodeled acceleration. Lab 08 measured Eq. (6) against a target that actually burns and found it *flatly excluded the truth for hours after a real burn*: conservatism ratio $0.5\times$ at both 2 h and 6 h — the cone claimed half the width the evidence demanded. The fix adds an impulse term growing linearly at the plausible burst Δv,

$$w(\Delta t) \;=\; w_0 + \sigma_v \Delta t + \tfrac{1}{2}\, a_{\max}\, \Delta t^2 \;+\; \Delta v_{\text{burst}}\, \Delta t, \tag{7}$$

and shipped as a prerequisite of fire control, with the lab's dishonesty table reproduced verbatim as a regression test.

Lab 06 caught the older sampled closest-approach scan stepping clean through a periapsis: 380,845 km reported versus 157,866 km true — a $2.4\times$ error at exactly the moment a player cares — and priced the fix. The closed-form segment minimum of Eq. (5) is that fix, and ordnance hit resolution inherits it.

The point for this venue: the curriculum is not documentation *about* the engine; it is an adversarial test suite the engine must survive, written to be read.

7.3 Fire control as the shooting method

A firing solution is a two-point boundary value problem: leave the muzzle now, occupy the predicted target point at t_{hit}. The unknowns are launch bearing θ and ejection charge q; the residual is the miss vector at arrival,

$$\mathbf{R}(\theta, q) \;=\; \mathbf{x}\big(t_{\text{hit}};\, \theta, q\big) \;-\; \hat{\mathbf{x}}_{\text{tgt}}\big(t_{\text{hit}}\big), \tag{8}$$

where $\mathbf{x}(t_{\text{hit}}; \theta, q)$ is obtained by one full flight of the real integrator of §2 — no closed-form ballistics, no cheating. The solver is a damped 2×2 Newton iteration,

$$\begin{pmatrix}\theta\\ q\end{pmatrix}_{m+1} = \begin{pmatrix}\theta\\ q\end{pmatrix}_{m} - \lambda\, J_m^{-1}\, \mathbf{R}_m, \qquad J_m = \left.\frac{\partial \mathbf{R}}{\partial(\theta, q)}\right|_m, \tag{9}$$

with J_m estimated by finite differences (each column one more integrator flight) and $\lambda \in (0, 1]$ a damping factor that backs off when a step would overshoot. This is the shooting method, verbatim, from any numerical-methods course — and the game plays it back honestly: the gun deck replays the iteration trace one Newton step per beat (CALCULATING FIRING SOLUTION...), locks at $T-60$ s, slews the hull, fires, and returns the helm — the Norden-bombsight beat, earned rather than faked.

Dispersion is not a tuning knob. The shot's uncertainty *is* the target track's cone half-width at arrival,

$$\sigma_{\text{hit}} \;=\; w_{\text{tgt}}\big(t_{\text{hit}}\big) \tag{10}$$

with w_{tgt} from Eq. (7), so fire-control quality *is* track quality: telescope work at the Sensors desk directly buys tighter shots at the gun deck — one weapon system distributed across two stations. Each locked solution prints its "gunner's lesson" (lead angle, time of flight): the solver as flight instructor, teaching the same intuition manual intercepts demand.

7.4 The lab-to-game pipeline: every number a probe printed

Fire control (§7.3) is the sharpest instance of a general method, not a one-off. The lab course is a working R&D bench: when the game needs a new behavior, a lab *measures* it first against the shipping engine, and only then does the game ship the measured number. The pattern is worth stating plainly because it is the paper's most reusable idea — *measure in a lab, then ship the measured number* — and the newest labs exist because a playtest exposed a mechanic the game was faking. Labs 21–30, by name and one-line finding:

- **Lab 21, "The Commuter"** (*reserved*): an Earth↔Mars Persephone cycler computed honestly in the rails, to land beside the free-return lesson.
- **Lab 22, "The air brake"**: the one new Core ingredient the flight assists needed — an exponential atmosphere with a single ballistic-coefficient knob; the Apollo skip's capture-without-burn-up corridor is ∼20 km of periapsis altitude wide, and a fuel-out arrival is turned bound ($114 \to 7\,R_J$) on drag alone.
- **Lab 23, "The moon run"**: the autopilot learns orbital mechanics — Wednesday's reset loop burned 677 pulses fighting Saturn; planning in the giant's frame (a 36.8 h transfer window whose porkchop floor *is* Hohmann) flies the same hop for **96 pulses**, a $7.1\times$ cut.
- **Lab 24, "The last mile"**: "almost there" is its own orbital problem — point-and-throttle chasing costs 4.0 km/s and arrives too hot to stop; 1960s change-your-period rendezvous doctrine closes a 92,640 km along-track gap for **2 pulses**, and `TransferPlanner` returns the whole cheaper-vs-sooner trade table.
- **Lab 25, "The tide that takes it back"**: the autopilot learns to *keep* an orbit. Swept in the live field, Luna and Titan have a stable park core but *Enceladus has none* — its Hill sphere is only 3.8 body radii, so an unkept park crashes within half a day. The keeping bill, priced in the game's own pulses: Enceladus **27 p/day**, Luna **2**, Titan **3** — the exact numbers the armed autopilot now quotes (§5).

- **Lab 26, "The soft arrival"** (*reserved*): Titan aerocapture as an arrival mode, composing Labs 22 and 23.
- **Lab 27, "The getaway"**: the wolves' honesty contract. Flown through the real thrust-only pursuit law, the killer identity $u^2/2a$ makes a grab earnable only inside ∼the 300,000 km catch radius at modest flee speed — a clean pursuit cliff past which the closest pass screams through at 24–55 km/s and cannot grab; the three measured escapes (a Jupiter sling donating 12 km/s, a 1,567 m/s heat-bleed skim, a phasing juke that voids the firing solution) become the `PursuitOdds` the BUSTED pop-up reads.
- **Lab 28, "The pump crawl"**: can the ship still reach a fuel pump? Depots never ride an ordinary moon, so the reach to the nearest pump is 29 pulses from a Titan doorstep — already beyond the old flat 45-pulse reserve, the fuel-starve bug in one number. It ships `FuelReachability.Assess`, a well-aware Comfortable/Thin/CannotReachAPump verdict, and it also became the BUSTED encounter's mercy law (never confiscate below reach-a-pump).
- **Lab 29, "The harbor pattern"**: what a safe arrival looks like. Flying a spread of inbound trajectories measures two harbor doors — a station clamp bubble that bills a pulse per ∼1% of world speed you arrive hot with, and a moon door (the 949 km Hill sphere) where a too-hot fall genuinely impacts. The corridor is a constant-time-to-go glideslope, shipped as `ApproachCorridor`, the seam the banner's NEXT row and the tutorial narration speak.
- **Lab 30, "The mass-driver timetable"**: the owner's canon (Luna lobbing compute-core pods) becomes measured physics. A pod has zero maneuver budget, so its future is a closed-form Kepler conic; above the 2.376 km/s lunar-escape floor a retrograde lob threads perihelion toward the Mercury compute yards, and the repeating cadence is a bus timetable read off the rail — half the pods always already in flight.

The through-line: five of these labs (23–25, 27, 29) were commissioned by a specific playtest failure and returned a specific constant — 96 pulses, 27 p/day, a 300,000 km cliff, a 949 km door — that the game now displays to the player as the honest cost of what they are doing. The curriculum is the game's R&D department, and its output is the game's numbers.

7.5 When the probe method left physics

The most surprising development in the lab course is that the method stopped being about orbital mechanics. Labs 32–44 include a class of lesson that measures the *game software* with the same instrument the earlier labs pointed at gravity, and each one exists because the owner said a version of the same sentence: *this keeps biting us; find a way to catch it in CI*.

- **Lab 34, "The unclickable lifeline"**: after a playtest in which the rescue affordance was barely clickable at exactly the moment a stranded player needed it, this lab measures the *interface* honestly — treating a critical control's reachability under duress as a quantity a probe can print, not a matter of taste.
- **Lab 41, "Can you get there from here?"**: written after the owner stood in a derelict that had been sealed in half — two barricades spanning the spine corridor cut off the manifest, four compartments and the entire aft end. The probe is an A* flood over generated interiors, wired into CI, and it is the direct ancestor of the reachability guarantee the underground complex now ships with (§5.6).
- **Lab 44, "A lab about the lab"**: the owner's own joke made real. The A* audit can prove *that* a room is unreachable; this lab asks whether anything can say *why* — the diagnosis problem behind a defect in which rooms were drawn and offered an interaction on 35 of 94 generated floors while every test stayed green.

The generalization is the one we did not anticipate when we wrote the pipeline down: the lab-to-game loop is not a physics technique. It is a technique for any property the team keeps asserting and cannot see — and level topology and interface reachability turned out to be exactly such properties, with exactly the same failure mode (a confident claim, a green suite, and a player standing in front of a wall).

8 Story QA: Testing What the Suite Cannot Reach

Determinism makes physics testable, and §7 argues the curriculum is an adversarial suite for the engine. Neither helps with content. A deterministic test can prove that a rule computes the right number; it cannot prove that a room is enterable, that a card appears at the right beat, or that a lift is a way *home*. The practices in this section are what we grew when content volume overtook the suite's reach, and we report them because they are the least transferable-looking part of the project and turned out to be the most.

8.1 The cheat-start law

The governing sentence is written in the source: *a scene nobody can reach on demand is a scene that ships broken*. Every scene, arc state, mission leg, death cause, and ground situation is reachable directly from a URL query parameter parsed in a single boot loop — 31 such parameters at this writing, each documented in the testing guide's appendix *in the same pull request that adds the feature*. The corollary is a QA instruction rather than an engineering one: if an arc has no quick start, saying so is itself a finding, and it gets filed. The payoff is that a reviewer can be standing inside any scene in the game in one navigation, which is what makes

review-by-playing survive content scale — without it, the cost of *reaching* a scene silently becomes the reason nobody checks it.

8.2 A guard must be proven red

The rule we would most like other teams to steal is small: *before a regression guard ships, revert the fix and watch the guard fail*. It exists because it caught the lead. A guard written for a level-generation defect passed against the *broken* generator, because it asserted the shape of the symptom rather than the law that had been violated. The generalization is that a test's honesty is a property of the world handed to it, not of its assertions: a correct assertion evaluated against a world that cannot distinguish pass from fail is a green test that asserts nothing.

8.3 Five named bug classes

Naming a recurring defect converts it from a surprise into a checklist item, and the project now keeps five names in a QA handoff document. They are, with the evidence that earned each its name:

1. **Unaudited client geometry literals** — numbers typed straight into a renderer that nothing derives and nothing checks. Found wrong three times out of three.
2. **A constant governing the wrong thing** — canonically, a *moon* constant governing a *ship*. Four occurrences.
3. **The simulation does one thing, the sentence or the drawn shape says another** — three in a single afternoon, every one found by playing and none by testing.
4. **One source consumed in the wrong order** — "a list built by appending is not a list in order." This one sealed thirty-five floors' worth of doorways while the entire suite stayed green.
5. **A green test that asserts nothing** — three independent instances found by three workers in three areas in one afternoon: a guard handed a world in which no body could build anything, and a threshold placed so that it selected every case.

8.4 Playing finds what reasoning and tests cannot

The clearest single datum in this section is a defect in which a lift only went *down*. It survived three of the lead's own pull requests and an A* reachability audit, for a reason worth stating precisely: *a reachability audit proves you can reach the lift; it never proves the lift is a way home*. The owner's method — boot every scene and check that all the parts are in the right place — caught it in one sitting. Nearly every expensive defect in this project has had the same signature: invisible to reasoning, invisible to the deterministic suite, and obvious on sight.

8.5 The art pipeline

Content at this cadence needed pictures at this cadence. The project's illustrations are generative, and the pipeline that keeps them honest is deliberately boring: each set of images has a *manifest* document in the repository (eight of them at this writing — bars, wrecks, the surface, the catch encounter, the underground, the two arcs, and the visions) naming every intended image, its prompt intent, and the code site that will display it. The game now ships 139 generated paintings, and the discipline that matters is a guard rather than a process: the suite asserts that every art filename referenced in code resolves to a file that exists, so the failure mode of generative content — a beautiful card with a broken image in it — is a build failure rather than a playtest report.

9 Experience Report: a Human PO and an AI Head Coder

9.1 Roles

The owner sets vision, plays every build, files corrections in plain language ("the sensors are off by default — why?"), and approves pull requests, often from a phone. The AI is a *head coder*, not a single programmer: it decomposes the owner's intent into lanes, writes each lane a coordination brief, dispatches an implementation agent per lane, reviews and integrates the results, verifies live in the owner's own browser, writes the PR narratives, and runs the merge train. Review is by playing, not by reading diffs alone.

9.2 Content per active day, honestly counted

We deliberately do not report a session speed record. The metric that matters for a game is how much *content* lands, and the honest denominator is *active coding days*, not wall-clock days: the project ran over a calendar span that included travel breaks with no merges at all. Counting merged pull requests by their squash-commit `(#N)` suffix on `main`[1] gives **424 merged PRs** over **24 active days — 17.7 PRs per active day** — inside a 33-day calendar window. The distribution (Table 1) shows both the ramp and the honest gaps.

Volume is not the claim; the claim is that it was *reviewed* volume. Every PR was played on a fresh build, and the suite grew alongside it. The growth is itself the paper's sharpest evidence that content, not physics, became the dominant cost: the deterministic Core suite stood at 787 facts on 18 July — immediately after the window the previous draft reported — 1,539 on 26 July, and 2,278 facts plus 68 theories (238 inline cases) at this writing, some 2,500 executed Core cases beside 79 client tests and a UI gate, running in roughly twenty-two

[1] Method: squash merges leave no merge commits, so `git log -merges` finds nothing. We instead count first-parent commits on `main` whose subject ends in a `(#N)` pull-request suffix, one per merged PR, and group them by author date: `git log -date=short -pretty='%ad %s' origin/main | grep -E '\(#[0-9]+\)$'`. Days with zero such commits are idle days and are excluded from the active-day count.

Date	PRs	Date	PRs	Date	PRs
Jul 02	1	Jul 14	17	Jul 27	16
Jul 03	23	Jul 15	14	Jul 28	24
Jul 04	25	Jul 16	18	Jul 29	19
Jul 05	32	Jul 17	30	Jul 30	28
Jul 06	11	Jul 18	42	Jul 31	7
Jul 07	2	Jul 19	30	Aug 01	14
Jul 08	4	Jul 20	22	Aug 02	32
Jul 13	2	Jul 26	5	Aug 03	6
Idle: Jul 09–12, Jul 21–25				**Total**	**424**

Table 1: Merged PRs per active day, counted once per distinct PR number across the release branches. 424 PRs / 24 active days = 17.7 per active day; the two gaps are travel breaks, excluded from the denominator.

minutes. The lab course grew from 30 lessons to 41 over the same stretch. Almost none of that tripling is new integrator code; it is guards for arcs, ground, cards, and reachability.

9.3 Orchestration: one lead, many agents, parallel worktrees

The throughput above is not one model typing faster; it is one *lead* model coordinating several implementation agents at once. Each agent works in its own isolated git worktree, so lanes never share a working tree and cannot clobber each other mid-edit. The lead writes each lane a coordination brief with explicit ownership boundaries — which files a lane may touch, and the one-line-append-at-a-marked-anchor rule for the shared hotspot files — then merges integration-branch-first: results land on an integration branch and are played there before any button touches `main`. When two lanes conflict, the conflict is sent *back to its author agent* to resolve, because that agent holds the context the merge does not. The discipline is what lets review-by-playing scale: the lead's attention is spent on intent and integration, not on typing every line.

Adversarial review is part of the loop, and it earns its keep by catching real bugs — including the lead's own. Two representative saves: the lead once hand-stitched a CSS change with an unbalanced brace, which no compiler exists to catch (CSS has none) and which only a live bench flight revealed — now a brace-balance check after any hand-edited CSS is standing law, and the bench flight is mandatory. And on another defect the lead proposed a root-cause hypothesis that a review lane *refuted*, returning the true root cause instead. Being honest about these — an AI lead is wrong in specific, ordinary ways — is what makes the method credible rather than a demo.

9.4 The live-browser loop: reading the owner's own tab

The tightest feedback path in the project is the lead driving the owner's *actual* running browser tab during a playtest. Two examples show the range. In the first, the owner revealed the Derelict Roadster but could not click it; the lead read the live tab, root-caused a class filter in the object picker that excluded the Roadster's category, shipped the fix to `main`, and the owner continued *the same mission* minutes later — triage-and-fix inside one sitting, no repro build required. In the second, a "phantom Dock button" appeared to lie about being dockable; the lead read the numbers off a *paused* frame's screenshot and found the presentation, not the simulation, at fault — the same class of perception bug as the earlier "three target locks," where the physics was right and the readout wrong. The screenshot-evidence loop — decide from the frame's actual printed numbers, not from what the UI seems to say — is how those get caught.

9.5 Working agreements that made the pace safe

- **Determinism is law** — it is what makes review-by-playing sound, because what the reviewer plays is what the tests tested.
- **Core-first with tests**: every mechanic lands as a pure rule with a deterministic test before any UI touches it.
- **Worktree isolation & merge-train hygiene**: one agent per isolated worktree; one new file per parallel lane; one-line appends at marked anchors in the shared hotspot file; integration-branch-first before any merge button.
- **The bench flight is not optional**: it is the only compiler some bugs (a CSS brace, a perception mismatch) will ever meet.
- **No unverified claims**: every sentence in a PR body is verified in a live browser before it is written.

9.6 Failure gallery (honest)

The interpreted-WASM freeze class (hit twice); the destroyed-canvas class — a Blazor conditional unmounting a canvas leaves the renderer holding a dead context (found three times, then prevented by convention); the perception-bug class, where the simulation was right and the presentation lied ("three target locks"; the phantom Dock button); the lead's own unbalanced CSS brace, caught only by a live bench flight and now guarded by a standing brace check; a lead root-cause hypothesis refuted by its own review lane; transient Pages deploy failures and the retry playbook; and a stacked-PR merge-train pitfall — deleting a merged base branch auto-closes its children — recovered live and folded into the plan document's procedure.

9.7 What the AI could not do

Choose what the game should feel like. Every mechanic in §4–§7 traces to an owner sentence written in plain language — the ship's voice, orbit-keeping as law, GTA-style BUSTED, buried treasure, the favor bank. The method's throughput is the owner's taste executed quickly, not replaced.

10 Related Work

Kerbal-genre orbital games adopt patched conics — piecewise two-body arcs with sphere-of-influence handoffs — trading multi-attractor truth for closed-form speed; we take the opposite stance and integrate everything, accepting Eq. (3)'s cost to keep flybys honest. The educational-games literature on "stealth learning" anticipates our disguise strategy; our addition is making the curriculum executable against the shipping engine. Determinism-first architectures descend from lockstep RTS netcode traditions. The fragment-assembly narrative of §6 sits in the environmental-storytelling and "audio-log" tradition, but differs in its gating: the reveal is withheld behind a *quorum* of independently-sourced fragments rather than a sequence, and the arcs' convergence is a threshold crossing rather than an authored beat, so its timing is a property of how a particular player played. Prior AI-pair-programming experience reports describe a programmer and an assistant; we report a different split — *product owner and head coder* — in which the human writes no code at all and the AI is not one coder but a lead orchestrating several implementation agents across isolated worktrees (§9.3).

11 Limitations and Future Work

The simulation is a 2D ecliptic plane with on-rails circular ephemerides. Multiplayer is archived pending the economy's server-side move. Persistence now exists for the *life* but not the physics: the vault (§5) durably stores relationships, balances, caches, and maps, while the mid-flight world state is still a fresh start — deliberately, so a save is a pirate's biography, not a checkpoint. Three lab slots remain reserved (the Persephone cycler, the Titan soft arrival, and one more), and the Ancients' auto-plot deliberately rations automation to protect the pedagogy.

The narrative work of §6 carries its own honest limitations. Both arcs are *assembly* complete and *destination* incomplete: the fragments, gates, and convergence all run, but the journey to arc one's ice moon is a one-time window that is hooked rather than flown, and the sanity-shock constants the reveals are meant to spend are declared and not yet consumed. The convergence card, as filed, spends both arcs' truths as an announcement at three assembled fragments per side — before either capstone is earned — which is a pacing defect we have recorded rather than hidden. And arc one's discoverability problem (§6) is open: the longest-prepared story in the game is currently invisible until a player trips over it.

12 Summary

SpaceSails demonstrates that one uncompromising property — bit-determinism — can be spent three ways at once: as an engineering tool (replay, lockstep, testability), as a game-design tool (honest instruments whose admissions of ignorance are mechanics), and as a pedagogical tool (a lab course that is also an adversarial regression suite). The engine's best fixes — the impulse-aware cone of Eq. (7), the closed-form no-tunneling test of Eq. (5) — were forced by its own curriculum, and its flagship mechanic is a numerical method played at the gun deck. On that same bench the game grew a voice, an orbit-keeping autopilot, a GTA-style catch encounter, and a buried-treasure economy — every displayed number first measured by a probe. It then grew in a direction the first draft of this paper did not anticipate: outward from the cockpit onto the ground, from hiding loot to detective work to walking a facility (§5.6), and into two long arcs whose truths are assembled by the player rather than told (§6) — at which point the honesty law had to be re-earned for content that no integrator can check, in the form of a cheat-start law, guards proven red before they ship, and five named bug classes (§8). The most portable finding of the second half is that the lab-to-game probe method was never about physics: pointed at level topology and interface reachability, it caught the same class of confident, green, wrong (§7.5). Around that core, a two-person human-PO / AI-head-coder team — one lead orchestrating parallel implementation agents — sustained 424 reviewed, merged PRs across 24 active coding days (17.7 per active day) without lowering the review bar, because determinism made playing the build a sound review procedure. We believe both halves — the simulation-as-curriculum design and the orchestrated PO/head-coder process — are repeatable, and we offer the repository itself as the evidence.

Acknowledgments

SpaceSails was built as a holiday project, and it was possible at this cadence because of Anthropic's generous usage allowance for the Claude models through Claude Code — 424 reviewed pull requests of head-coder and implementation-agent work, live-browser verification included. That allowance is, in a real sense, the project's funding agency, and we acknowledge it as such. Anthropic neither directed nor reviewed this work. SpaceSails is a personal project of the first author and is not affiliated with Efima or Anthropic.

Reproducibility. Everything in this paper runs from the public repository: `dotnet test` for the evidence, `dotnet run -project labs/NN-... -c Release` for every printed number, and the game itself at the GitHub Pages deployment (`https://esoinila.github.io/SpaceSails-play`).